Lecture Slip, CS 235
Nov 12, 2026, Thursday
Expression Evaluator

Name:	
EmpID:	

1. **Trace `toPostfix`.** Convert `8 - 2 * (1 + 3)` to postfix with the shunting-yard code from class. The tokens are `8`, `-`, `2`, `*`, `(`, `1`, `+`, `3`, `)`. Rows 1 to 6 are done. Complete the remaining rows: the **operator stack** (bottom to top) and the **output** after each token.

Token	Operator stack	Output so far	Action
`8`	(empty)	`8`	number goes to the output
`-`	`-`	`8`	stack empty, push
`2`	`-`	`8 2`	number goes to the output
`*`	`- *`	`8 2`	`prec(-) < prec(*)`: nothing to pop, push
`(`	`- * (`	`8 2`	push
`1`	`- * (`	`8 2 1`	number
`+`			
`3`			
`)`			
end			

   Now evaluate your postfix with a stack of values. List the value stack (bottom to top) right after the `*` is processed. What is the final value?

2. **Operand order.** In `evalPostfix`, an operator takes the top of the value stack as `right`, pops it, then takes the next one as `left`, and pushes `left - right` for `-`.

   (a) Evaluate the postfix `9 4 -`. What are `right`, `left`, and the result?

   (b) A student writes `right` and `left` in the opposite order (so the code computes `right - left`). What does `9 4 -` give now? Would the same swap change the result of `9 4 +`?

3. **Two stacks, no postfix.** `evalInfix` keeps a stack of values and a stack of operators. It reads the tokens of `2 + 3 * 4` one at a time.

   (a) After **all** tokens have been read, but **before** the final clean-up loop, what is on each stack (bottom to top)?

   (b) The clean-up loop then runs. Show the value stack after each of the two operators is applied.

   (c) When `toPostfix` pops an operator from its stack, it sends it to the output. What does `evalInfix` do with that operator instead?

4. **Where does the error come from?** Assume `tokenize` has already produced the tokens. Both inputs are bad. For each, say which part of `toPostfix` throws, and why.

   (a) `1 + 2)` (b) `(1 + 2`

5. **Pace survey.** Please help me set the pace of the course. Circle one.

Too **slow** (I can go faster)	**About right** (normal pace)	Too **fast** (please slow down)

6. **Topics to clarify.** Check **all** topics you would like me to explain again next class.

- ☐ Finishing `toPostfix` (`(` and `)`)
- ☐ `evalPostfix` and the operand order
- ☐ Two stacks: `evalInfix` and `applyTop`
- ☐ Digits to a number (`num * 10 + (ch - '0')`)
- ☐ Linear search and its cost
- ☐ Binary search code (`lo`, `hi`, `mid`)
- ☐ Why binary search is $O(\log n)$
- ☐ Other: _____

7. Any other questions about today's class? If so, please provide details. Thank you.